Bangladesh University of Business and Technology

LAB REPORT

Experiment No : 03
Course Title : Algorithms Lab
Course Code : CSE 232
Experiment Name: Lab Assignment 03

Submitted By:

Name : Kawsar Hoque
ID No : 20255103270
Intake : 55
Section : 7
Program : B.Sc. Engg. in CSE

Submitted To:

Name : Sumona Yeasmin

(Lecturer)
Dept. of CSE
Bangladesh University of Business & Technology

Date of Submission : 17/06/2026

1) Merge Sort

Start here × mergesort270.cpp ×

```cpp
#include <iostream>
using namespace std;

void merge(int A[], int l, int m, int h)
{
    int n1 = m - l + 1;
    int n2 = h - m;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++)
        L[i] = A[l + i];
    for (int j = 0; j < n2; j++)
        R[j] = A[m + 1 + j];

    int i = 0, j = 0, k = l;

    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            A[k] = L[i];
            i++;
        }
        else
        {
            A[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1)
    {
        A[k] = L[i];
        i++;
        k++;
    }
    while (j < n2)
    {
        A[k] = R[j];
        j++;
        k++;
    }
}

void mergesort(int A[], int l, int h)
{
    if (l < h)
    {
        int m = (l + h) / 2;
        mergesort(A, l, m);
        mergesort(A, m + 1, h);
        merge(A, l, m, h);
    }
}

int main()
{
    int n;
    cout << "Size of array: ";
    cin >> n;

    int arr[n];

    cout << "Enter your array: ";
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }

    if (n > 0)
    {
        mergesort(arr, 0, n - 1);
    }

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Output:

"C:\Users\Kawsar Hoque\Desl

```
Size of array: 5
Enter your array: 8 3 9 5 4
Sorted array: 3 4 5 8 9

Process returned 0 (0x0)   execution time : 19.484 s
Press any key to continue.
```

2) Quick Sort - Pivot as First Element (Using For Loop)

Start here × quicksort_first_pivot_for270.cpp ×

```cpp
#include <iostream>
using namespace std;

int partition(int A[], int l, int h)
{
    int pivot = A[l];
    int j = l;

    for (int i = l + 1; i <= h; i++)
    {
        if (A[i] < pivot)
        {
            j++;
            swap(A[i], A[j]);
        }
    }
    swap(A[l], A[j]);
    return j;
}

void quickSort(int A[], int l, int h)
{
    if (l < h)
    {
        int pivot = partition(A, l, h);
        quickSort(A, l, pivot - 1);
        quickSort(A, pivot + 1, h);
    }
}

int main()
{
    int n;
    cin >> n;
    int arr[n];
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    if (n > 0)
        quickSort(arr, 0, n - 1);
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}
```

"C:\Users\Kawsar Hoque\Desl

```
5
9 8 7 6 5
5 6 7 8 9
Process returned 0 (0x0)   execution time : 21.705 s
Press any key to continue.
```

3) Quick Sort - Pivot as First Element (Using While Loop)

Start here × quicksort_while270.cpp ×

```cpp
#include <iostream>
using namespace std;

int partition(int A[], int l, int h)
{
    int i = l;
    int j = h;
    int pivot = A[l];

    while (i < j)
    {
        while (i <= h && A[i] <= pivot)
            i++;
        while (A[j] > pivot)
            j--;
        if (i < j)
            swap(A[i], A[j]);
    }
    swap(A[l], A[j]);
    return j;
}

void quickSort(int A[], int l, int h)
{
    if (l < h)
    {
        int pivot = partition(A, l, h);
        quickSort(A, l, pivot);
        quickSort(A, pivot + 1, h);
    }
}

int main()
{
    int n;
    cin >> n;
    int arr[n];
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    if (n > 0)
        quickSort(arr, 0, n - 1);
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}
```

"C:\Users\Kawsar Hoque\Desl

```
5
9 8 4 6 11
4 6 8 9 11
Process returned 0 (0x0)   execution time : 14.033 s
Press any key to continue.
```

4) Quick Sort - Pivot as First Element (Using Do-While Loop)

Start here | quicksort_first_pivot_for270.cpp

```cpp
#include <iostream>
using namespace std;

int partition(int A[], int l, int h)
{
    int pivot = A[l];
    int j = l;

    for (int i = l + 1; i <= h; i++)
    {
        if (A[i] < pivot)
        {
            j++;
            swap(A[i], A[j]);
        }
    }
    swap(A[l], A[j]);
    return j;
}

void quickSort(int A[], int l, int h)
{
    if (l < h)
    {
        int pivot = partition(A, l, h);
        quickSort(A, l, pivot - 1);
        quickSort(A, pivot + 1, h);
    }
}

int main()
{
    int n;
    cin >> n;
    int arr[n];
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    if (n > 0)
        quickSort(arr, 0, n - 1);
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}
```

"C:\Users\Kawsar Hoque\Desl

```
5
9 8 7 6 5
5 6 7 8 9
Process returned 0 (0x0)   execution time : 21.705 s
Press any key to continue.
```

5) Quick Sort - Pivot as Last Element

Start here × quicksort_last_pivot270.cpp ×

```cpp
#include <iostream>
using namespace std;

int partition(int A[], int l, int h)
{
    int pivot = A[h];
    int i = l - 1;

    for (int j = l; j < h; j++)
    {
        if (A[j] < pivot)
        {
            i++;
            swap(A[i], A[j]);
        }
    }
    swap(A[i + 1], A[h]);
    return i + 1;
}

void quickSort(int A[], int l, int h)
{
    if (l < h)
    {
        int pivot = partition(A, l, h);
        quickSort(A, l, pivot - 1);
        quickSort(A, pivot + 1, h);
    }
}

int main()
{
    int n;
    cin >> n;
    int arr[n];
    for (int i = 0; i < n; i++)
        cin >> arr[i];
    if (n > 0)
        quickSort(arr, 0, n - 1);
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}
```

"C:\Users\Kawsar Hoque\Desl

```
8
4 2 2 8 3 3 1 5
1 2 2 3 3 4 5 8
Process returned 0 (0x0)   execution time : 2.652 s
Press any key to continue.
```